



COEP TECHNOLOGICAL UNIVERSITY, PUNE

Wellesley Road, Shivajinagar, Pune-411 005

Assignment 6

Aim: To Implement a priority-based task scheduler using Treap data structure. Where each task has a unique taskID as the BST key and a priority as the Max-heap priority, so that the highest-priority task can be executed first while maintaining BST properties.

Algorithm:

1. rotateRight()

- Store $root \rightarrow left$ as newRoot
- Set $root \rightarrow left = newRoot \rightarrow right$
- Set $newRoot \rightarrow right = root$.
- return newRoot;

2. rotateLeft()

- Store $root \rightarrow right$ as newRoot
- Set $root \rightarrow right = newRoot \rightarrow left$;
- Set $newRoot \rightarrow left = root$;
- Return newRoot.

3. Insert task - insert()

- If $root == NULL$:
 - create and return a new node.
- Compare the new task.id with $root \rightarrow task.id$
- If $task.id < root \rightarrow task.id$:
 - Recursively insert into the left subtree.
 - If the new left child has greater priority than the root, perform a right rotation.



- If $task_id > root \rightarrow task_id$:
 - Recursively insert into the right subtree.
 - If the new right child has greater priority ^{than} $root$,
 - perform a left Rotation.
- If $task_id == root \rightarrow task_id$:
 - Report that the task already exists.
- Return the $root$.

4. Delete task — `deleteNode()`

- If $root == NULL$, return $NULL$;
- Compare $task_id$ with $root \rightarrow task_id$.
- If $task_id < root \rightarrow task_id$:
 - Recursively delete from left subtree.
- If $task_id > root \rightarrow task_id$:
 - Recursively delete from right subtree.
- If task is found:
 - case1: No left child
 - Store the right child.
 - Free the current node.
 - Return the right child.
 - case2: No right child
 - Store the left child.
 - Free the current node.
 - Return the left child.



COEP TECHNOLOGICAL UNIVERSITY, PUNE

Wellesley Road, Shivajinagar, Pune-411 005

case

case 3: Both children exist

- compare the priorities of the left and right children
- If left child has higher priority:
 - Perform a right Rotation
 - Recursively delete the task from the right subtree.

- Otherwise:

- Perform a left Rotation
- Recursively delete the task from the left subtree.

- Return the Root.

5. Execute Highest Priority task - executeTask()

- If the tree is empty:
 - Display "No tasks available"
 - Return NULL;
- Display the task id and priority of the root.
- Delete the root task using deleteNode();
- Return the updated root.

6. Search Task - searchTask()

- If root == NULL return NULL
- If root->taskId == taskId:
 - return the current Node.
- If taskId < root->taskId:
 - Recursively search the left subtree.
- otherwise
 - Recursively search the right subtree.

COEP TECHNOLOGICAL UNIVERSITY, PUNE

Wellesley Road, Shivajinagar, Pune-411 005

7. Inorder Traversal - `inorder()`

- If `root == NULL`, return
- Traverse the left subtree.
- Print current node's task ID and priority.
- Traverse the right subtree.

8. Preorder Traversal - `preorder()`

- If `root == NULL`, return
- print current node's Task ID and priority.
- Traverse the left subtree.
- Traverse the right subtree.

9. Free Tree - `freeTree()`

- If `root == NULL`, return
- Recursively free left subtree.
- Recursively free right subtree.
- free current node.

Conclusion: We have successfully implemented the Tree operation.